Search Framework Architecture

Components

Module	Role
`search.py`	Orchestrator – `_TransformGraph`, serial search loop, progress reporting
`compile.py`	`CompilationPool`, NKI compilation, hardware benchmarking, result types
`report.py`	Live JSON report with depth distributions and per-variant results

Main Thread runs a serial loop: pick a random frontier node, apply one transform (`expand_one`), verify numerical correctness, lower to NKI, and submit to `CompilationPool`. The serial path takes ~0.2s per variant even for 1601-stmt programs – verification and lowering are fast.

CompilationPool runs `roll_loops` + `compile_to_neff` via neuronxcc in a `ProcessPoolExecutor` with ~191 workers. Compilation runs concurrently while the main thread keeps expanding. After the search loop finishes, `pool.wait_all()` blocks for remaining compilations.

Hardware Benchmark distributes compiled NEFFs across up to 128 Neuron cores for latency measurement.

Runtime Profile (1024x1024 FP16 matmul, trn2.48xlarge)

Metric	Value
Total wall time	~2 min
Root program	1,601 stmts, ~4,096 transforms/node
Qualifying variants	128 compiled, 128 benchmarked, 0 failures
Best variant	137 us, 19.9% MFU

Phase	Time	Detail
Search loop (serial)	~29s	128 expand + verify + lower + submit
NEFF compilation (parallel)	~63s	128 variants on 191 CPU workers, overlaps with search

Phase	Time	Detail
Hardware benchmark	~25s	128 NEFFs on 128 Neuron cores

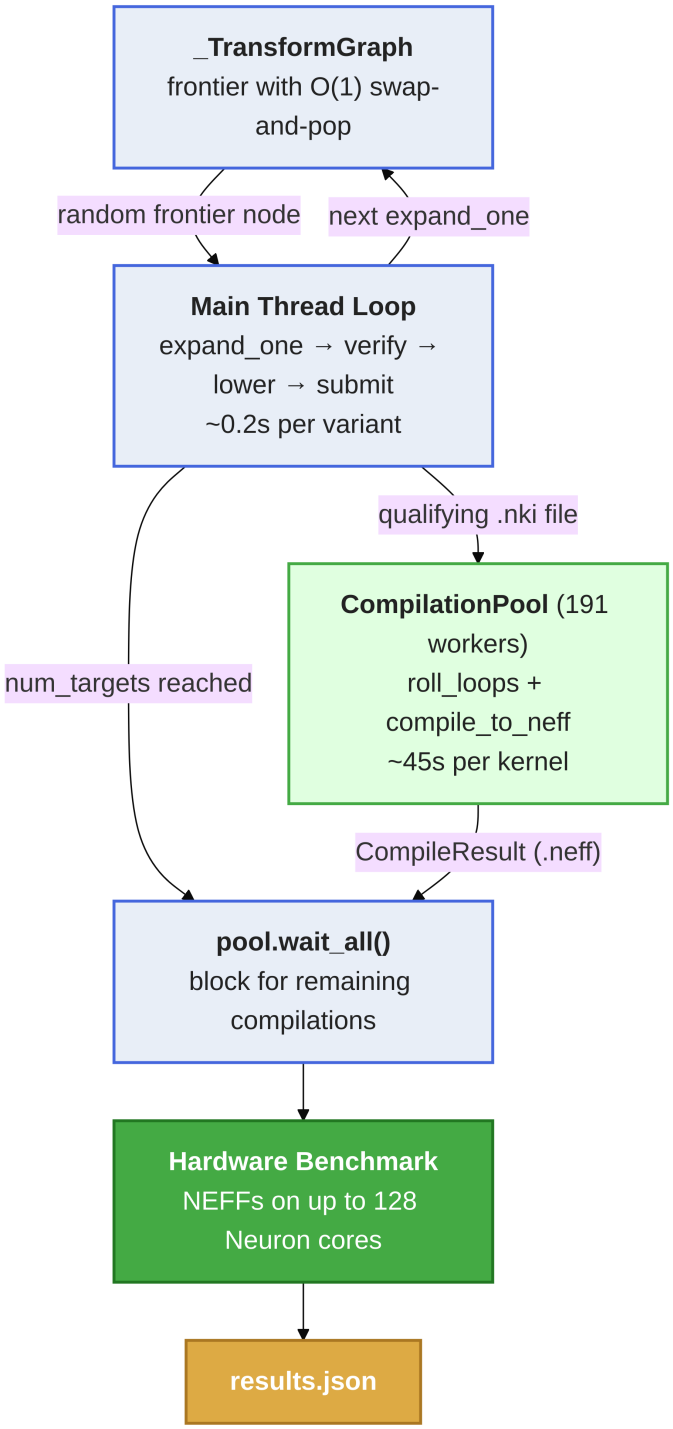

Figure 1: Search framework architecture